SYSTEM CALL MONITOR

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

G . SAHITHI	**2520030499**
B . JASMITHA	**2520030557**

Under the Esteemed Guidance of

Y . Harika
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **SYSTEM CALL MONITOR** is a record of Bonafide work of G. SAHITHI- 2520030499 , B. JASMITHA submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

G . SAHITHI	**2520030499**
B. JASMITHA	**2520030557**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"System Call Monitor"** is a bonafide work done and submitted by G . Sahithi - 2520030499 , B . Jasmitha - 2520030557

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Y . Harika , Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

G . SAHITHI **2520030499**

B.JASMITHA **2520030557**

Abstract

The System Call Monitor is a Linux-based C programming project developed to provide a practical understanding of how user-space applications communicate with the operating system kernel through system calls. The system demonstrates essential file and process management operations using system calls such as open(),read(),write(),close(), fork(), exec(),and wait()/waitpid().

The project creates and manipulates files, creates and manages processes, executes external programs, and synchronizes parent and child processes. It also implements proper system-call return-value checking and uses errno to identify and handle runtime errors. The strace utility is integrated to trace the system calls generated during program execution, allowing users to observe the sequence of interactions between an application and the Linux kernel.

By combining implementation with system-call tracing and monitoring, the project provides a hands-on demonstration of user-space to kernel-space interaction, process management, file operations, error handling, and operating-system services. It is intended as an educational tool for students to understand the practical working of Linux system calls beyond theoretical concepts.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8 - 13
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		14-25
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		26-29
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		30-31
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		32-34
		5.1 Overall System Architecture	
		5.2System Workflow	

		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**35-47**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**48-50**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**51-52**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**53-54**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An Operating System (OS) acts as an interface between the user and the computer hardware. It manages system resources such as memory, files, processes, and input/output devices. Applications running on a computer require operating system services to perform various operations. However, user applications cannot directly access most hardware and protected system resources. They communicate with the operating system kernel through a mechanism known as **System Calls**.

System calls provide an interface between user-level applications and the Linux kernel. They allow programs to perform essential operations such as creating files, reading and writing data, creating processes, executing programs, and synchronizing processes.

In Linux, commonly used system calls include `open()`, `read()`, `write()`, `close()`, `fork()`, `exec()`, and `wait()` or `waitpid()`. These system calls play an important role in file management and process management.

The **System Call Monitor** is a Linux-based project developed using the C programming language to demonstrate the practical working of system calls. It performs file operations, creates and manages processes, executes external programs, and demonstrates process synchronization.

The project also uses the `errno` mechanism for identifying errors during system-call execution and the `strace` utility for tracing system calls generated by the program.

Through this project, users can observe how applications interact with the Linux kernel and understand the importance of system calls in operating system functionality.

1.2 Problem Statement

Understanding how applications interact with an operating system is a fundamental concept in Systems programming and Operating systems. Applications cannot directly access most system resources; instead, they communicate with the Linux kernel through system calls to perform operations such as creating and accessing files, managing processes, executing programs, and synchronizing tasks.

However, students often learn these concepts primarily through theory, making it difficult to visualize what happens between a user-space application and the kernel. The sequence of system calls, their return values, error conditions, and the transition from user space to kernel space are not always easy to observe during normal program execution.

To address this gap, there is a need for a practical system that demonstrates commonly used Linux system calls and allows their execution to be monitored. The proposed System Call Monitor provides a C-based environment for performing file operations using open(), read(), write(), and close(), along with process management using fork(), exec(), and wait()/waitpid(). The system also incorporates return - value validation and errno for proper error handling.

Furthermore, the project uses strace to trace the system calls generated during execution, enabling users to examine how applications request services from the Linux kernel. This provides a practical understanding of system-call execution, process creation and synchronization, file management, error handling, and the interaction between user-space programs and kernel services.

1.3 Scope of the Project

The scope of the System Call Monitor project is to demonstrate and monitor essential Linux system calls using the C programming language in a Linux environment.

The project focuses on the following areas:

File Management: Demonstrates creating, opening, reading, writing, and closing files using `open()`, `read()`, `write()`, and `close()`.

Process Creation: Demonstrates the creation of a child process using the `fork()` system call and explains the relationship between parent and child processes.

Program Execution: Uses the `exec()` family of functions to replace the existing process program with another executable program.

Process Identification: Uses `getpid()` and `getppid()` to display the Process ID and Parent Process ID.

Process Synchronization: Demonstrates how the parent process waits for the child process to complete using `wait()` or `waitpid()`.

Error Handling: Checks system-call return values and uses `errno` to identify and report errors.

System Call Monitoring: Uses the `strace` utility to trace system calls generated during program execution and observe their sequence.

User-Kernel Interaction: Provides an understanding of how user-space applications request services from the Linux kernel through system calls.

1.4 Objectives of the Project

The main objectives of the System Call Monitor project are:

To understand the concept and importance of Linux system calls.

To demonstrate file operations using `open()`, `read()`, `write()`, and `close()`.

To understand process creation and management using `fork()`.

To demonstrate the execution of external programs using the `exec()` family of functions.

To implement process synchronization using `wait()` and `waitpid()`.

To identify processes using `getpid()` and `getppid()`.

To implement proper error checking for system-call operations.

To understand Linux error handling using `errno`.

To use the `strace` utility to trace and monitor system calls.

To observe the interaction between user-space applications and kernel services.

To gain practical knowledge of file management, process management, and system programming in Linux.

1.5 Importance of the Application

1. Practical Understanding of System Calls:
The application helps users understand how system calls work by implementing commonly used Linux system calls in C rather than learning them only through theoretical concepts.

2. Understanding File Operations:
It demonstrates how applications create, access, read, write, and close files using system calls. This helps users understand how file management is handled by the operating system.

3. Process Management Knowledge:
The project demonstrates how new processes are created, how programs are executed, and how parent and child processes interact.

4. Process Synchronization:
By using `wait()` and `waitpid()`, the application explains how a parent process can wait for the completion of its child process, helping users understand process synchronization.

5. Error Detection and Handling:
The use of return-value checking and `errno` helps identify errors during system-call execution and improves understanding of error-handling mechanisms.

6. System Call Tracing:
The integration of `strace` allows users to observe the system calls generated during program execution. It helps connect program instructions with the services provided by the Linux kernel.

7. Understanding User-Space and Kernel-Space Interaction:
The project provides a clear understanding of how user applications request operating system services through system calls.

8. Academic and Practical Learning:
The application acts as a learning tool for students studying operating systems, Linux programming, and system-level application development.

1.6 Target Users / Audience

1. Computer Science and Engineering Students:
Students studying Operating Systems and Systems Programming can use this application to understand the practical implementation of system calls.

2. Beginners in Linux Programming:
Students and beginners learning C programming in a Linux environment can understand how applications interact with operating system services.

3. Operating System Learners:
Individuals interested in learning about file management, process creation, program execution, and process synchronization can use the project as a practical demonstration.

4. Programming and Academic Instructors:
Teachers and instructors can use the project as a demonstration tool to explain Linux system calls and their role in operating system functionality.

5. System Programming Enthusiasts:
Individuals interested in Linux system programming can explore the implementation and tracing of system calls to improve their understanding of low-level operating system operations.

CHAPTER 2

2.1 Operating System Concepts Related to the Project

An Operating System (OS) is system software that manages computer hardware and software resources and provides services to application programs. It acts as an interface between users, applications, and hardware.

The System Call Monitor project demonstrates several important operating system concepts through practical implementation using the C programming language in a Linux environment.

The major operating system concepts related to this project are explained below.

1. System calls

System calls provide an interface through which user-level programs request services from the operating system kernel. They allow applications to perform operations such as file access, process creation, memory allocation, and communication.

Examples:

`open()` – Opens or creates a file.

`read()` – Reads data from a file or pipe.

`write()` – Writes data to a file or pipe.

`close()` – Closes an open file descriptor.

`fork()` – Creates a child process.

`waitpid()` – Waits for a child process to complete.

`mmap()` – Maps memory into a process address space.

`munmap()` – Releases a memory mapping

2. File Management

File management is an important responsibility of an operating system. It allows applications to create, access, modify, and manage files stored in the file system.

In this project, file operations are demonstrated using `open()`, `read()`, `write()`, and `close()`.

The program accepts text from the user, writes it into a file named `system_call_sample.txt`, and reads the saved content back. It also checks whether the file exists and retrieves information such as file size, inode number, permissions, and owner ID.

3. Process Management

Process management involves creating, executing, scheduling, and terminating processes in an operating system.

A process is a program that is currently being executed. Linux provides system calls to create and manage processes.

In this project, `fork()` creates a child process from the parent process. The child process displays its Process ID (PID) and Parent Process ID (PPID). The `execlp()` function is then used to execute the Linux `ls` command. The parent process uses `waitpid()` to wait for the child process to finish.

4. Inter-Process Communication (IPC)

Inter-Process Communication (IPC) allows two or more processes to exchange information and coordinate their activities.

Linux supports several IPC mechanisms, including pipes, shared memory, message queues, and sockets.

This project demonstrates IPC using an unnamed pipe. The parent process sends a message through the pipe, and the child process reads and displays the received message.

The `pipe()`, `read()`, and `write()` operations are used to demonstrate communication between processes.

5. Signal Handling

Signals are software notifications used by the operating system or other processes to notify a process about particular events.

Linux provides different signals such as `SIGINT`, `SIGTERM`, and `SIGUSR1`.

In this project, the `SIGUSR1` signal is used to demonstrate signal handling. The `sigaction()` function registers a signal handler, and `raise()` generates the signal. The handler updates a variable of type `sig_atomic_t` to indicate that the signal has been received.

This demonstrates how a running program can respond to signals.

6. Memory Management

Memory management is the process of allocating, using, and releasing memory for running programs.

Linux provides memory-management facilities that allow applications to request memory mappings and release them when they are no longer required.

This project demonstrates memory management using `mmap()` and `munmap()`.

The program allocates a memory region of 4096 bytes, stores a text message in the allocated memory, displays the stored data, and releases the memory using `munmap()`.

7. File System Management

A file system organizes and stores data on storage devices. It maintains information about files, directories, permissions, ownership, and storage locations.

In this project, the `access()` function checks whether the demonstration file exists, while the `stat()` function retrieves file information.

The program displays the file size, inode number, file permissions, and owner user ID.

8. Thread Management and Synchronization

Threads are smaller units of execution within a process. Multiple threads can execute concurrently while sharing the same process memory.

Thread synchronization ensures that multiple threads access shared resources safely and prevents problems such as race conditions.

This project uses POSIX threads (Pthreads) to create two threads. Both threads increment a shared counter 10,000 times. A mutex lock is used to protect the shared counter during updates.

The functions `pthread_create()`, `pthread_mutex_lock()`, `pthread_mutex_unlock()`, and `pthread_join()` are used to demonstrate thread creation, synchronization, and completion

9. Error Handling

Error handling is important in system programming because system calls and operating system operations may fail due to different conditions.

In this project, return values are checked to identify unsuccessful operations. The `errno` variable and `strerror()` function are used to identify and display error messages.

A common error-reporting function named `print_error()` is implemented to display the operation that failed and the corresponding error description.

2.2 Existing Methods / Algorithms

Traditional methods of understanding operating system concepts mainly involve studying theoretical explanations, executing individual Linux commands, and writing separate programs for different system operations.

Although these methods help students understand individual concepts, they may require switching between different programs and commands to observe the working of multiple operating system functionalities.

The commonly used existing methods are explained below.

1. Manual Linux Command Execution

Linux terminal commands are commonly used to perform file and process management operations.

Commands such as `ls`, `cat`, `ps`, and `stat` allow users to view files, display file contents, monitor processes, and retrieve file information.

However, these commands generally show the final results of operations rather than providing a complete demonstration of how different system calls are implemented within a C program.

2. Individual C Programs

Another existing method is developing separate C programs for each operating system concept.

For example:

One program for file operations.

Another program for process creation.

A separate program for IPC.

Individual programs for memory management and signals.

Although this method is useful for learning individual concepts, managing multiple programs can make demonstrations less organized.

3. Direct System Call Implementation

Linux system calls can be demonstrated by writing C programs that directly invoke operating system interfaces.

Functions such as `open()`, `read()`, `write()`, `fork()`, and `waitpid()` are commonly used to understand how applications request operating system services.

This approach provides practical knowledge but requires proper error checking and an understanding of Linux programming.

4. Command-Line System Call Tracing

The `strace` utility is commonly used to trace system calls made by a running program.

It displays information about system calls, their arguments, return values, and errors.

This method helps developers understand the interactions between user-space applications and the Linux kernel. However, interpreting trace output may be difficult for beginners without a practical demonstration of the corresponding operations.

5. Traditional Process and Thread Synchronization

Process and thread synchronization are commonly demonstrated using operating system mechanisms such as pipes, mutex locks, and process waiting functions.

Separate examples can be used to understand communication between processes and the coordination of multiple threads.

However, studying these mechanisms independently may make it difficult to understand their relationship with other operating system concepts.

Limitations of Existing Methods

The following limitations are identified in traditional approaches:

Different operating system concepts are often demonstrated using separate programs.

Students may find it difficult to connect theoretical concepts with actual Linux implementation.

Manual command execution does not provide a unified demonstration of different system operations.

Error handling may not be properly demonstrated in basic examples.

Understanding IPC and thread synchronization requires separate practical implementations.

System-call tracing requires additional knowledge to interpret the generated output.

Switching between multiple programs and commands can make demonstrations less organized.

2.3 Proposed Approach

The proposed approach is to develop a menu-driven **System Call Monitor** using the C programming language in a Linux environment.

The application combines multiple operating system demonstrations into a single program. It allows users to select and execute individual modules through a simple terminal-based menu.

Each module demonstrates a specific operating system concept using appropriate Linux system calls and APIs.

The proposed system consists of the following modules:

1. File Input and Output Module

Accepts text input from the user.

Creates or opens a file using `open()`.

Writes the entered text using `write()`.

Reads the saved content using `read()`.

Closes the file using `close()`.

2. Process Management Module

This module demonstrates process creation and execution.

Creates a child process using `fork()`.

Displays the PID and PPID.

Executes the Linux `ls` command using `execlp()`.

Uses `waitpid()` to synchronize the parent and child processes.

3. Inter-Process Communication Module

This module demonstrates communication between two processes.

Creates an unnamed pipe using `pipe()`.

Creates a child process using `fork()`.

The parent process writes a message into the pipe.

The child process reads the message.

Displays the received message on the terminal.

4. Signal Handling Module

This module demonstrates how a program handles signals.

Registers a signal handler using `sigaction()`.

Generates the `SIGUSR1` signal using `raise()`.

Handles the received signal through a custom signal handler.

Updates a signal-status variable to indicate successful signal handling.

5. Memory Management Module

This module demonstrates dynamic memory mapping.

Allocates a memory region of 4096 bytes using `mmap()`.

Stores a text message in the allocated memory.

Displays the stored message.

Releases the allocated memory using `munmap()`.

6. File System Information Module

This module retrieves information about the demonstration file.

Checks file existence using `access()`.

Retrieves file information using `stat()`.

Displays file size, inode number, permissions, and owner user ID.

7. Thread Concurrency Module

This module demonstrates multithreading and synchronization.

Creates two threads using `pthread_create()`.

Both threads increment a shared counter.

Uses a mutex lock to protect the shared counter.

Uses `pthread_join()` to wait for both threads to complete.

Displays the final counter value..

8. Menu-Driven Execution Module

The main function provides a menu through which users can select the required demonstration.

The available options are:

File Input and Output

Process Management

Inter-Process Communication

Signal Handling

Memory Management

File System Information

Thread Concurrency

Run All Demonstrations

Exit

The application also validates user input and displays an error message when an invalid menu choice is entered.

Error Handling and Monitoring

The proposed system includes error checking for important operations. It uses return-value validation, `errno`, and `strerror()` to identify and report errors.

Additionally, the `strace` Linux utility can be used externally to trace the system calls generated during program execution. This helps users observe the system-level operations performed by the application.

Overall, the proposed approach provides an integrated environment for understanding file management, process management, IPC, signal handling, memory management, file system information, and thread synchronization.

2.4 Advantages of the Proposed Approach

The proposed System Call Monitor provides several advantages over traditional methods of demonstrating operating system concepts.

1. Unified Demonstration

Multiple operating system concepts are implemented within a single C program. Users do not need to execute separate programs for every demonstration.

2. Easy-to-Use Menu Interface

The menu-driven interface allows users to select the required module easily. It also provides an option to execute all demonstrations together.

3. Practical Understanding of System Calls

The application demonstrates the practical use of Linux system calls such as `open()`, `read()`, `write()`, `fork()`, `waitpid()`, `mmap()`, and `munmap()`.

4. Better Understanding of Process Management

The project demonstrates parent-child process relationships, process creation, program execution, and process synchronization.

5. Demonstration of Inter-Process Communication

The use of unnamed pipes helps users understand how two processes exchange information through a communication channel.

6. Signal Handling Demonstration

The application demonstrates how a process registers and handles signals using `sigaction()` and `raise()`.

7. Memory Management Understanding

The project explains memory allocation and deallocation through `mmap()` and `munmap()`, helping users understand memory management at the system-programming level.

8. Thread Synchronization

The application demonstrates concurrent execution using two threads and protects shared data using mutex synchronization.

9. Improved Error Handling

The use of return-value checking, `errno`, and `strerror()` helps identify errors and provides meaningful error messages during execution.

10. File System Information Retrieval

The application retrieves useful file details, including file size, inode number, permissions, and ownership.

11. System Call Monitoring Support

The application can be executed with the `strace` utility to observe system calls and understand the interactions between the application and the Linux kernel.

12. Educational Value

The project provides a practical learning environment for students studying Operating Systems, Linux programming, and Systems Programming. It connects theoretical concepts with their actual implementation in C.

CHAPTER 3

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-Core Processor
RAM	4 GB
Storage	10 GB free space
System Architecture	64-bit processor
Display	Standard Monitor
Keyboard	Standard Keyboard

3.2 Software Requirements

Software / Tool	Purpose
Ubuntu Linux	Provides the environment for executing Linux system calls.
Windows Subsystem for Linux (WSL)	Provides a Linux environment within Windows.
C Programming Language	Used to implement system-call demonstrations.
GCC Compiler	Compiles the C source code into an executable program.
POSIX Threads (Pthreads)	Supports thread creation and synchronization.
Linux System Libraries	Provides access to Linux and POSIX APIs.
Linux Terminal / Bash	Used to compile, execute, and test the application.
strace	Used to trace and monitor system calls during program execution.
Git and GitHub	Used for source-code version control and project collaboration

3.3 Operating System Environment – Ubuntu / WSL

The System Call Monitor application is developed in a Linux-based environment using Ubuntu. It provides the required interfaces for file operations, process management, IPC, memory management, signal handling, and thread synchronization.

The project can also run through Windows Subsystem for Linux (WSL), which allows Linux applications and tools to run on Windows without a separate Linux computer

Ubuntu / WSL supports the following project operations:

File handling using `open()`, `read()`, `write()`, and `close()`.

Process creation and synchronization using `fork()` and `waitpid()`.

Command execution using `execlp()`

Inter-process communication using pipes.

Signal handling using `sigaction()` and `raise()`.

Memory allocation using `mmap()` and `munmap()`.

File information retrieval using `access()` and `stat()`.

Thread creation and synchronization using Pthreads and mutex locks.

Linux Commands Used

Command	**Purpose**
pwd	Displays the current directory.
ls	Lists files and directories.
gcc	Compiles the C program.
./program	Executes the program.
strace	Traces system calls.
clear	Clears the terminal.

3.4 Development Tools and Compilers

The System Call Monitor uses various tools and technologies for coding, compilation, execution, and testing.

1. C Programming Language

C is used to implement file operations, process management, IPC, signal handling, memory management, and thread synchronization.

2. GCC Compiler

GCC compiles the C source code into an executable program.

Example:
```
gcc system_call_monitor.c -o system_call_monitor -pthread
```

The `-pthread` option enables thread support.

3. Linux Terminal / Bash

The terminal is used to navigate directories, compile and execute programs, view outputs, and run Linux commands.

4. strace Utility

`strace` traces system calls and signals during program execution, helping users observe interactions between the application and the Linux kernel.

Example:
```
strace ./system_call_monitor
```

5. POSIX Threads Library

Pthreads supports concurrent execution and thread synchronization. The project uses:

`pthread_create()` – Creates threads.

`pthread_join()` – Waits for threads to finish.

`pthread_mutex_lock()` and `pthread_mutex_unlock()` – Provide synchronization.

6. Git and GitHub

Git tracks source-code changes, while GitHub stores the project repository and supports team collaboration.

Linux Header Files

The project uses the following header files:

Header File	Purpose
stdio.h	Input and output
stdlib.h	General utilities
string.h	String operations
unistd.h	POSIX system interfaces
fcntl.h	File operations
errno.h	Error handling
sys/types.h	System data types
sys/wait.h	Process synchronization
sys/stat.h	File information
sys/mman.h	Memory mapping
signal.h	Signal handling
pthread.h	Thread management

CHAPTER 4

TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is the primary programming language used in this project. It provides access to Linux system interfaces and supports low-level operations such as file handling, process creation, memory allocation, and signal handling.

4.2 GCC Compiler

The GNU Compiler Collection (GCC) is used to compile the C source code and generate an executable file.

Example command:

```
gcc system_call_monitor.c -o system_call_monitor -pthread
```

The `-pthread` option enables POSIX thread support.

4.3 Linux System Calls

Linux system calls allow applications to request services from the operating system kernel. The project demonstrates:

File Operations: `open()`, `read()`, `write()`, `close()`

Process Management: `fork()`, `waitpid()`, `execlp()`

IPC: `pipe()`

Signal Handling: `sigaction()`, `raise()`

Memory Management: `mmap()`, `munmap()`

File Information: `access()`, `stat()`

4.4 POSIX Threads / Synchronization

The POSIX Threads (Pthreads) library is used to demonstrate concurrent execution. Two threads increment a shared counter, while mutex locks protect the shared data from race conditions.

Functions used include `pthread_create()`, `pthread_join()`, `pthread_mutex_lock()`, and `pthread_mutex_unlock()`.

4.5 Linux Commands and Utilities

Linux commands and utilities are used to compile, execute, and monitor the application.

Command / Utility	Purpose
pwd	Displays the current directory
ls	Lists files and directories
gcc	Compiles the program
./system_call_monitor	Executes the program
strace	Traces system calls
clear	Clears the terminal

4.6 Other Tools / Libraries Used

Ubuntu / WSL: Provides the Linux environment for development and execution.

strace: Helps monitor system calls generated during program execution.

errno: Identifies errors during system operations.

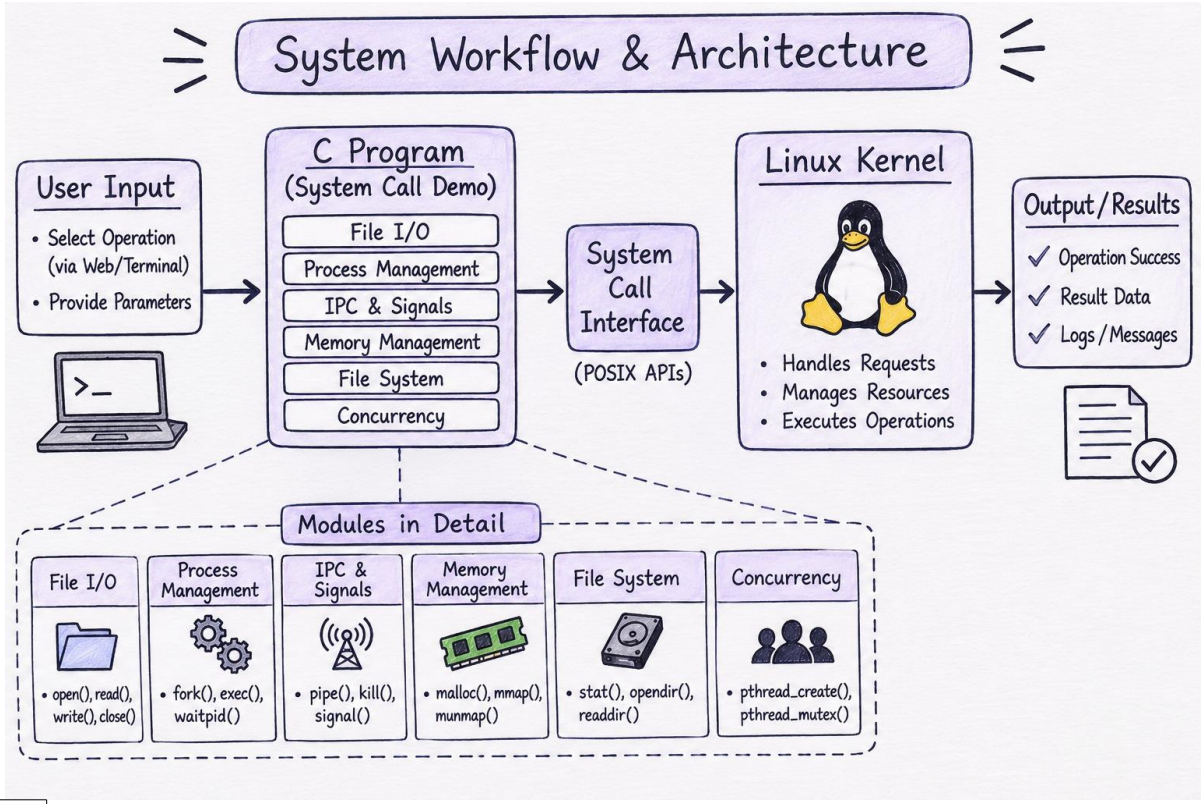
Git: Tracks changes in the source code.

GitHub: Stores the project repository and supports collaboration.

Linux Header Files: Provide the required declarations and interfaces for system calls, memory management, signals, and threads.

CHAPTER 5
SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture



The image part with relationship ID rId7 was not found in the file.

5.2 System Workflow

The program starts and displays the main menu.

The user selects an operation.

The program validates the entered choice.

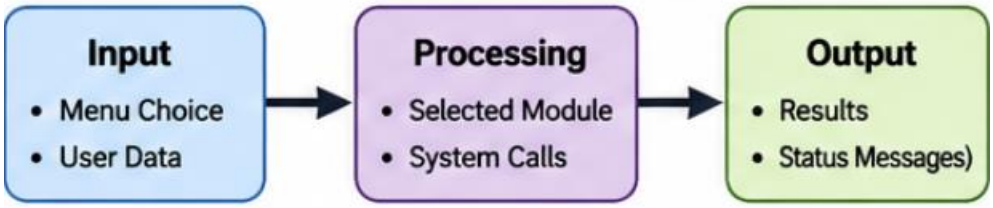
The corresponding module is executed.

The module performs the required system operations.

The result or error message is displayed.

The menu is displayed again until the user selects Exit.

5.3 Flowchart / Data Flow



5.4 Module Description

The System Call Monitor consists of seven major modules.

1. File I/O Module

Uses `open()`, `write()`, `read()`, and `close()` to create a file, write user input, read its contents, and display the result.

2. Process Management Module

Uses `fork()` to create a child process, `execlp()` to execute an external command, and `waitpid()` to synchronize the parent and child processes.

3. Inter-Process Communication Module

Uses `pipe()` to establish communication between parent and child processes. The parent sends a message, which the child reads and displays.

4. Signal Handling Module

Uses `sigaction()` to register a signal handler and `raise()` to generate `SIGUSR1`. The handler records that the signal was received.

5. Memory Management Module

Uses `mmap()` to allocate memory and `munmap()` to release it after storing and displaying sample data.

6. File System Information Module

Uses `access()` and `stat()` to check file existence and retrieve details such as file size, inode number, permissions, and owner ID.

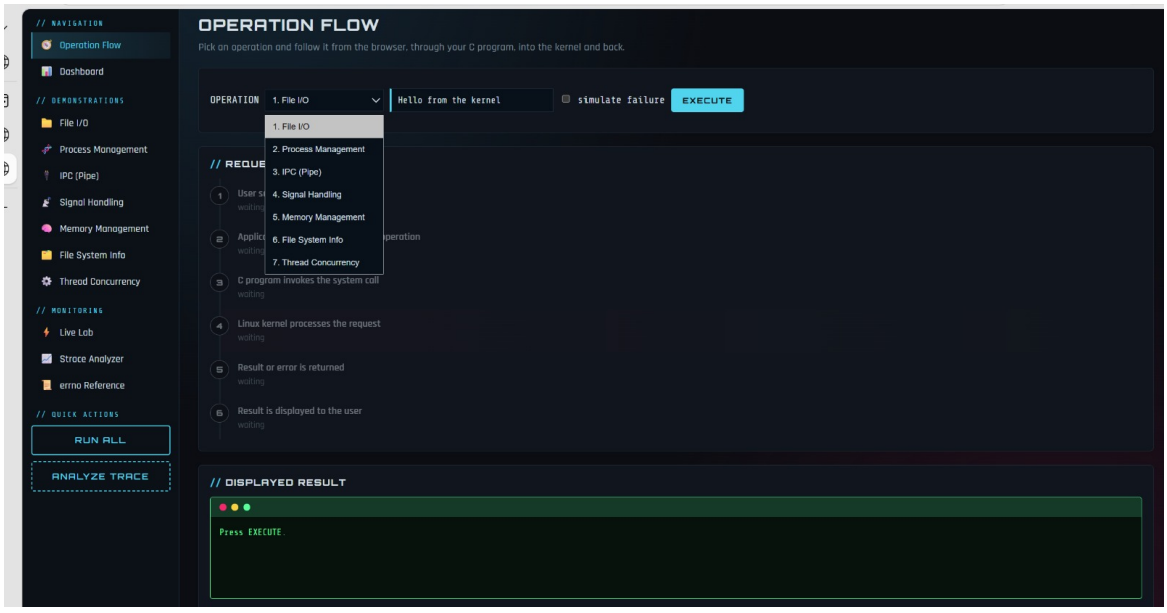
7. Thread Concurrency Module

Creates two threads using `pthread_create()`. Both threads increment a shared counter, while mutex locks ensure safe access to shared data. `pthread_join()` waits for both threads to finish.

5.5 Input and Output Design

The application uses a simple command-line interface for user interaction.

Input Design



Output Design

File creation, writing, and reading results.

Parent and child process details.

Messages exchanged through pipes.

Signal reception status.

Memory allocation and data display.

File metadata such as size and permissions.

Final shared counter value after thread execution.

Error messages when an operation fails.

CHAPTER 6
IMPLEMENTATION

6.1 Module-wise Implementation

CO	Course Outcome	Concepts / Functions Used in the Project
CO-1	Operating System Fundamentals	User/kernel interaction, Linux system calls, shell commands, strace
CO-2	Process Control	fork(), execlp(), waitpid(), _exit()
CO-3	Inter -Process Communication	Anonymous pipes using pipe(), read(), write(), signal handling using sigaction() and raise()
CO-4	Virtual Memory	Memory mapping using mmap() and memory release using munmap()
CO-5	File Systems	open(), read(), write(), close(), access(), stat(), inode information, file size and permissions
CO-6	Concurrency	POSIX threads, pthread_create(), pthread_join(), mutex locks, shared counter, race-condition prevention

6.2 Algorithm / Pseudocode

Algorithm: System Call Monitor

Start the program.

Initialize required variables and libraries.

Display the main menu.

Accept the user's choice.

Validate the entered choice.

Execute the corresponding module:

File I/O

Process Management

IPC

Signal Handling

Memory Management

File System Information

Thread Concurrency

Display the execution result or error message.

If the user selects Run All, execute all modules.

Repeat the menu until the user selects Exit.

Terminate the program.

Pseudocode:

```
START
  REPEAT
    DISPLAY menu
    READ user_choice

    IF choice is valid THEN
      SWITCH choice
        CASE 1: File I/O
        CASE 2: Process Management
        CASE 3: IPC
        CASE 4: Signal Handling
        CASE 5: Memory Management
        CASE 6: File System Information
        CASE 7: Thread Concurrency
```

```
            CASE 8: Run All Modules
            CASE 0: Exit
          END SWITCH
       ELSE
          DISPLAY invalid choice
       END IF
    UNTIL choice = 0
STOP
```

6.3 Core C / C++ Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <signal.h>
#include <pthread.h>

#define DEMO_FILE "system_call_sample.txt"
#define BUFFER_SIZE 256

volatile sig_atomic_t signal_received = 0;

pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
int shared_counter = 0;

/* Display system call errors */
void print_error(const char *operation)
{
   fprintf(stderr, "%s failed: %s\n", operation, strerror(err>
}

/* 1. File Input and Output */
void demo_file_io(void)
{
   char message[BUFFER_SIZE];
   char buffer[BUFFER_SIZE] = {0};

   printf("\n========== FILE I/O ==========\n");

   printf("Enter the text you want to write: ");

   if (fgets(message, sizeof(message), stdin) == NULL) {
```

```
        printf("Input error.\n");
        return;
    }

    int fd = open(DEMO_FILE, O_CREAT | O_WRONLY | O_TRUNC, 064>

    if (fd == -1) {
        print_error("open");
        return;
    }

    size_t length = strlen(message);
    size_t total_written = 0;

    while (total_written < length) {
        ssize_t written = write(fd, message + total_written,
                                length - total_written);

        if (written == -1) {
            print_error("write");
            close(fd);
            return;
        }

        total_written += (size_t)written;
    }
printf("\n[File I/O] Text written successfully using write>
    printf("[File I/O] Bytes written: %zu\n", total_written);

    if (close(fd) == -1) {
        print_error("close");
        return;
    }

    fd = open(DEMO_FILE, O_RDONLY);

    if (fd == -1) {
        print_error("open for reading");
        return;
    }

    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);

    if (bytes_read == -1) {
        print_error("read");
        close(fd);
        return;
    }

    buffer[bytes_read] = '\0';
```

```
    printf("[File I/O] Reading saved text:\n");
    printf("%s", buffer);

    close(fd);

    printf("[File I/O] File closed successfully.\n");
}

/* 2. Process Management */
void demo_process_management(void)
{
    printf("\n========== PROCESS MANAGEMENT ==========\n");

    pid_t pid = fork();

    if (pid < 0) {
        print_error("fork");
        return;
    }

    if (pid == 0) {
        printf("[Process] Child process created.\n");
        printf("[Process] Child PID: %d\n", getpid());
        printf("[Process] Parent PID: %d\n", getppid());

        printf("[Process] Executing ls command using exec().\n>

        execlp("ls", "ls", "-l", DEMO_FILE, (char *)NULL);

        print_error("execlp");
        _exit(EXIT_FAILURE);
    } else {
        int status;

        printf("[Process] Parent process PID: %d\n", getpid());
        printf("[Process] Waiting for child process.\n");

        if (waitpid(pid, &status, 0) == -1) {
            print_error("waitpid");
            return;
 }

        printf("[Process] Child process completed.\n");
    }
}

/* 3. Inter-Process Communication using Pipe */
void demo_ipc(void)
{
    int pipefd[2];
    char buffer[BUFFER_SIZE] = {0};
```

```
    printf("\n========== INTER-PROCESS COMMUNICATION =========>

    if (pipe(pipefd) == -1) {
        print_error("pipe");
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        print_error("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        return;
    }

    if (pid == 0) {
        close(pipefd[1]);
   ssize_t n = read(pipefd[0], buffer, sizeof(buffer) - 1>

        if (n == -1) {
            print_error("pipe read");
        } else {
            buffer[n] = '\0';
            printf("[IPC] Child received message: %s\n", buffe>
        }

        close(pipefd[0]);
        _exit(EXIT_SUCCESS);
    } else {
        const char *message =
            "Message sent from parent to child through a pipe.>

        close(pipefd[0]);

        if (write(pipefd[1], message, strlen(message)) == -1) {
            print_error("pipe write");
        } else {
            printf("[IPC] Parent sent message through pipe.\n">
        }

        close(pipefd[1]);
        waitpid(pid, NULL, 0);
    }
}

/* 4. Signal Handling */
void signal_handler(int signum)
{
    if (signum == SIGUSR1) {
```

```
  signal_received = 1;
    }
}

void demo_signals(void)
{
    struct sigaction sa;

    printf("\n========== SIGNAL HANDLING ==========\n");

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
        print_error("sigaction");
        return;
    }

    printf("[Signal] Sending SIGUSR1 signal.\n");

    if (raise(SIGUSR1) != 0) {
        print_error("raise");
        return;
    }

    if (signal_received) {
        printf("[Signal] SIGUSR1 received and handled successf>
    }
}
/* 5. Memory Management */
void demo_memory_management(void)
{
    size_t size = 4096;

    printf("\n========== MEMORY MANAGEMENT ==========\n");

    char *memory = mmap(NULL, size,
                        PROT_READ | PROT_WRITE,
                        MAP_PRIVATE | MAP_ANONYMOUS,
                        -1, 0);

    if (memory == MAP_FAILED) {
        print_error("mmap");
        return;
    }

    snprintf(memory, size,
            "Memory allocated using mmap() system call.");

    printf("[Memory] Memory allocated successfully.\n");
```

```
    printf("[Memory] Stored data: %s\n", memory);

    if (munmap(memory, size) == -1) {
        print_error("munmap");
        return;
    }

    printf("[Memory] Memory released using munmap().\n");
}

/* 6. File System Information */
void demo_filesystem(void)
{
    struct stat file_info;

    printf("\n========== FILE SYSTEM INFORMATION ==========\n">

    if (access(DEMO_FILE, F_OK) == 0) {
        printf("[File System] File exists: %s\n", DEMO_FILE);
    } else {
        print_error("access");
        return;
    }

    if (stat(DEMO_FILE, &file_info) == -1) {
        print_error("stat");
        return;
    }

    printf("[File System] File size: %ld bytes\n",
           (long)file_info.st_size);

    printf("[File System] Inode number: %ld\n",
           (long)file_info.st_ino);

    printf("[File System] File permissions: %o\n",
           (unsigned int)(file_info.st_mode & 0777));

    printf("[File System] Owner user ID: %ld\n",
           (long)file_info.st_uid);
}
/* 7. Thread Synchronization */
void *thread_function(void *arg)
{
    int i;

    (void)arg;

    for (i = 0; i < 10000; i++) {
        pthread_mutex_lock(&counter_mutex);
        shared_counter++;
```

```
        pthread_mutex_unlock(&counter_mutex);
    }

    return NULL;
}

void demo_concurrency(void)
{
    pthread_t thread1, thread2;

    printf("\n========== THREAD CONCURRENCY ==========\n");

    shared_counter = 0;

    if (pthread_create(&thread1, NULL, thread_function, NULL) >
        fprintf(stderr, "Thread 1 creation failed.\n");
        return;
    }

    if (pthread_create(&thread2, NULL, thread_function, NULL) >
        fprintf(stderr, "Thread 2 creation failed.\n");
        pthread_join(thread1, NULL);
 return;
    }

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("[Thread] Two threads executed successfully.\n");
    printf("[Thread] Final shared counter: %d\n", shared_count>
    printf("[Thread] Mutex synchronization completed.\n");
}

/* 8. Execute All Demonstrations */
void run_all_demonstrations(void)
{
    printf("\n\n=============================================\n");
    printf("     RUNNING ALL DEMONSTRATIONS\n");
    printf("=============================================\n");

    demo_file_io();
    demo_process_management();
    demo_ipc();
    demo_signals();
    demo_memory_management();
    demo_filesystem();
    demo_concurrency();

    printf("\n=============================================\n");
    printf("     ALL DEMONSTRATIONS COMPLETED\n");
    printf("=============================================\n");
```

```
}
/* Main Menu */
int main(void)
{
    int choice;
    char input[32];

    while (1) {
        printf("\n");
        printf("==============================================\n>
        printf("     SYSTEM CALL DEMONSTRATION SYSTEM\n");
        printf("==============================================\n>

        printf("1. File Input and Output\n");
        printf("2. Process Management\n");
        printf("3. Inter-Process Communication\n");
        printf("4. Signal Handling\n");
        printf("5. Memory Management\n");
        printf("6. File System Information\n");
        printf("7. Thread Concurrency\n");
        printf("8. Run All Demonstrations\n");
        printf("0. Exit\n");

        printf("----------------------------------------\n>
        printf("Enter your choice: ");

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        char *end;
        errno = 0;
        long selected = strtol(input, &end, 10);
  while (*end == ' ' || *end == '\t' ||
               *end == '\n' || *end == '\r') {
            end++;
        }

        if (errno != 0 || end == input || *end != '\0' ||
            selected < 0 || selected > 8) {
            printf("Invalid choice. Please enter 0 to 8.\n");
            continue;
        }

        choice = (int)selected;

        switch (choice) {
            case 1:
                demo_file_io();
                break;
```

```
            case 2:
                demo_process_management();
                break;

            case 3:
                demo_ipc();
                break;

            case 4:
                demo_signals();
                break;

            case 5:
                demo_memory_management();
                  break;

            case 6:
                demo_filesystem();
                break;

            case 7:
                demo_concurrency();
                break;

            case 8:
                run_all_demonstrations();
                break;

            case 0:
                pthread_mutex_destroy(&counter_mutex);
                printf("\nExiting System Call Demonstration Sy>
                return 0;
        }
    }

    pthread_mutex_destroy(&counter_mutex);
    return 0;
}
```

6.4 Linux System Calls / APIs Used

Function	Purpose
open()	Opens or creates a file
read()	Reads file or pipe data
write()	Writes data
close()	Closes a file descriptor
fork()	Creates a child process
waitpid()	Waits for a child process
pipe()	Creates a communication channel
sigaction()	Registers a signal handler
raise()	Generates a signal
mmap()	Maps memory
munmap()	Releases mapped memory
access()	Checks file accessibility
stat()	Retrieves file information

6.5 Process / Thread Implementation

Process Implementation

The process management module uses `fork()` to create a child process. The child executes the Linux `ls` command using `execlp()`, while the parent waits for its completion using `waitpid()`.

Thread Implementation

The concurrency module creates two threads using `pthread_create()`. Each thread increments a shared counter 10,000 times. The main thread uses `pthread_join()` to wait for both threads.

The expected final counter value is **20,000**.

6.6 IPC / Synchronization Implementation

Inter-Process Communication

The IPC module uses an unnamed pipe to exchange data between the parent and child processes.

The parent writes a message into the pipe.

The child reads the message.

The received message is displayed on the terminal.

Thread Synchronization

A mutex lock protects the shared counter from simultaneous modifications.

The functions `pthread_mutex_lock()` and `pthread_mutex_unlock()` ensure that only one thread updates the counter at a time.

6.7 Memory Management Implementation

The memory management module uses `mmap()` to allocate a memory region of 4096 bytes.

The program stores sample text in the allocated memory and displays it. After the operation, `munmap()` releases the allocated memory region.

This demonstrates dynamic memory mapping and memory release in Linux.

6.8 Optional User Interface Implementation

The project uses a **command-line interface (CLI)** instead of a graphical user interface.

The terminal displays a numbered menu that allows users to select individual demonstrations or execute all modules.

The interface accepts user input, displays execution results, and reports errors. This simple design makes the application easy to operate in the Ubuntu/Linux environment.

CHAPTER 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The C source code is compiled using the GCC compiler in the Ubuntu terminal.

Steps:

Open the Ubuntu terminal.

Navigate to the project directory.

Compile the C source file using GCC.

Generate the executable file.

Compilation command:

```
gcc main.c -o system_call_monitor -pthread
```

The `-pthread` option enables POSIX thread support required by the concurrency module.

If compilation is successful, the executable file is generated.



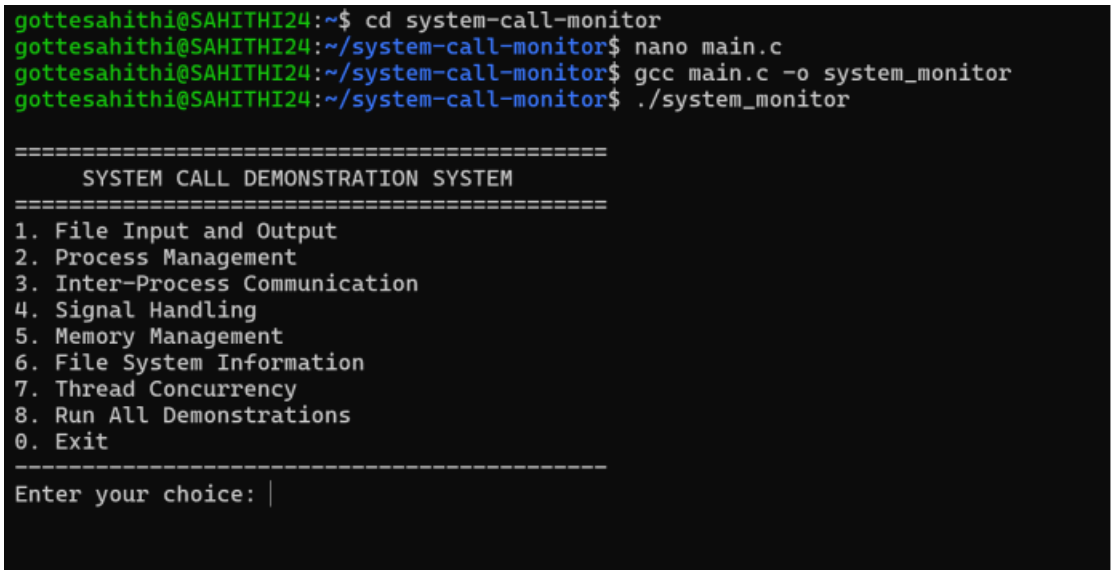
7.2 Execution Commands

Command	Purpose
pwd	Displays the current directory.
ls	Lists project files.
gcc main.c -o system_call_monitor -pthread	Compiles the program.
./system_call_monitor	Runs the application.
strace ./system_call_monitor	Traces system calls during execution.
clear	Clears the terminal.

7.3 Sample Inputs and Outputs

Module	Sample Input / Action	Expected Output
File I/O	Enter Hello Linux	File is created, written, and read successfully.
Process Management	Select option 2	Child process executes the ls command.
IPC	Select option 3	Parent sends a message and child displays it.
Signal Handling	Select option 4	SIGUSR1 signal is received and handled.
Memory Management	Select option 5	Memory is allocated, sample data is displayed, and memory is released.
File System Information	Select option 6	Displays file size, inode, permissions, and owner ID.
Thread Concurrency	Select option 7	Two threads execute and the final counter reaches 20,000.

7.4 Screenshots of Execution



```
gottesahithi@SAHITHI24:~/system-call-monitor$ nano main.c
gottesahithi@SAHITHI24:~/system-call-monitor$ gcc main.c -o system_monitor
gottesahithi@SAHITHI24:~/system-call-monitor$ ./system_monitor

==========================================
     SYSTEM CALL DEMONSTRATION SYSTEM
==========================================
1. File Input and Output
2. Process Management
3. Inter-Process Communication
4. Signal Handling
5. Memory Management
6. File System Information
7. Thread Concurrency
8. Run All Demonstrations
0. Exit
------------------------------------------
Enter your choice: 1

========== FILE I/O ==========
Enter the text you want to write: hello

[File I/O] Text written successfully using write().
[File I/O] Bytes written: 6
[File I/O] Reading saved text:
hello
[File I/O] File closed successfully.
```

```
gottesahithi@SAHITHI24:~/system-call-monitor$ nano main.c
gottesahithi@SAHITHI24:~/system-call-monitor$ gcc main.c -o system_monitor
gottesahithi@SAHITHI24:~/system-call-monitor$ ./system_monitor

==========================================
     SYSTEM CALL DEMONSTRATION SYSTEM
==========================================
1. File Input and Output
2. Process Management
3. Inter-Process Communication
4. Signal Handling
5. Memory Management
6. File System Information
7. Thread Concurrency
8. Run All Demonstrations
0. Exit
------------------------------------------
Enter your choice: 1

========== FILE I/O ==========
Enter the text you want to write: hello

[File I/O] Text written successfully using write().
[File I/O] Bytes written: 6
[File I/O] Reading saved text:
hello
[File I/O] File closed successfully.

------------------------------------------
Enter your choice: 2

========== PROCESS MANAGEMENT ==========
[Process] Parent process PID: 2168
[Process] Waiting for child process.
[Process] Child process created.
[Process] Child PID: 2173
[Process] Parent PID: 2168
[Process] Executing ls command using exec().
-rw-r--r-- 1 gottesahithi gottesahithi 6 Nov  1 18:57 system_call_sample.txt
[Process] Child process completed.

------------------------------------------
Enter your choice: 3

========== INTER-PROCESS COMMUNICATION ==========
[IPC] Parent sent message through pipe.
[IPC] Child received message: Message sent from parent to child through a pipe.

==========================================
0. Exit
------------------------------------------
Enter your choice: 4

========== SIGNAL HANDLING ==========
[Signal] Sending SIGUSR1 signal.
[Signal] SIGUSR1 received and handled successfully.

------------------------------------------
```

```
----------------------------------------
Enter your choice: 5

========== MEMORY MANAGEMENT ==========
[Memory] Memory allocated successfully.
[Memory] Stored data: Memory allocated using mmap() system call.
[Memory] Memory released using munmap().

========================================
```

```
----------------------------------------
Enter your choice: 6

========== FILE SYSTEM INFORMATION ==========
[File System] File exists: system_call_sample.txt
[File System] File size: 6 bytes
[File System] Inode number: 9045
[File System] File permissions: 644
[File System] Owner user ID: 1000

========================================
```

```
----------------------------------------
Enter your choice: 7

========== THREAD CONCURRENCY ==========
[Thread] Two threads executed successfully.
[Thread] Final shared counter: 20000
[Thread] Mutex synchronization completed.

========================================
     SYSTEM CALL DEMONSTRATION SYSTEM
========================================
1. File Input and Output
2. Process Management
3. Inter-Process Communication
4. Signal Handling
5. Memory Management
6. File System Information
7. Thread Concurrency
8. Run All Demonstrations
0. Exit
----------------------------------------
Enter your choice:
```

7.5 Demonstration of OS Concepts

OS Concept	Demonstration in the Project
System Calls	Interaction between the application and Linux kernel.
File Management	Creating, reading, and writing files.
Process Management	Creating and executing child processes.
IPC	Communication between processes using pipes.
Signal Handling	Generating and handling SIGUSR1.
Memory Management	Allocating and releasing memory using mmap() and munmap().
File System	Retrieving file metadata using stat().
Thread Management	Executing multiple threads concurrently.
Synchronization	Using mutex locks to protect shared data.
Error Handling	Identifying errors using errno and displaying error messages.

CHAPTER 8
CONCLUSION

8.1 Summary of the Project

The System Call Monitor is a menu-driven C application developed in Ubuntu/Linux to demonstrate important operating system concepts. It provides practical demonstrations of file operations, process management, inter-process communication, signal handling, memory management, file-system information, and thread synchronization.

The project uses Linux system calls and POSIX APIs to help understand how applications interact with operating system services.

8.2 Objectives Achieved

Demonstrated file operations using `open()`, `read()`, `write()`, and `close()`.

Implemented process creation and execution using `fork()` and `execlp()`.

Demonstrated process synchronization using `waitpid()`.

Implemented inter-process communication using anonymous pipes.

Demonstrated signal handling using `sigaction()` and `raise()`.

Implemented memory mapping using `mmap()` and `munmap()`.

Retrieved file information using `access()` and `stat()`.

Demonstrated thread synchronization using POSIX threads and mutex locks.

Used error handling and `strace` to observe system-level operations.

8.3 Results Achieved

The project was successfully implemented and tested in the Ubuntu/Linux environment. All seven demonstration modules perform their intended operations.

The application successfully creates and reads files, executes child processes, exchanges messages through pipes, handles signals, allocates and releases memory, retrieves file metadata, and performs concurrent operations.

The thread module produces an expected final counter value of 20,000, demonstrating successful mutex-based synchronization.

8.4 Skills and Learning Outcomes

Practical knowledge of Linux operating system concepts.

Understanding of system calls and kernel interaction.

C programming and Linux-based application development.

Process creation, execution, and synchronization.

Inter-process communication and signal handling.

Memory mapping and file-system operations.

Multithreading and synchronization using mutexes.

Compilation, debugging, and execution using GCC and Ubuntu.

System-call tracing using `strace`.

Teamwork, problem-solving, and technical documentation.

CHAPTER 9
REFERENCES

9.1 Books

Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 10th Edition, Wiley, 2018.

William Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition, Pearson, 2018.

W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3rd Edition, Addison-Wesley, 2013.

Michael Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.

9.2 Research Papers

Dennis M. Ritchie and Ken Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, Vol. 17, No. 7, pp. 365–375, 1974.

Edsger W. Dijkstra, "Solution of a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 8, No. 9, p. 569, 1965.

9.3 Linux / GNU Documentation

Linux man-pages Project – System Call and Library Function Documentation.

GNU Compiler Collection (GCC) Documentation.

Linux Kernel Documentation.

Linux `strace` Manual.

POSIX Threads (Pthreads) Documentation.

9.4 Online Resources

Linux man-pages: https://man7.org/linux/man-pages/

GNU GCC Documentation: https://gcc.gnu.org/onlinedocs/

Ubuntu Documentation: https://help.ubuntu.com/

strace Documentation: https://strace.io/

GitHub Repository: https://github.com/princyjasmitha202-sys/KLH_CSE_Y25_S9_13_System-Call-Monitor/tree/main